Sustainable Automated Paint Production

Industrial Automation Project

Alessandro Plantera, Veronica Consoli, Rubin Byrd Ronchieri

March 30, 2025

Contents

1 Introduction

This report presents the development and analysis of an automated control system for sustainable paint production. The objective is to achieve optimal mixing of primary colors while minimizing material waste and energy consumption. The system regulates pigment mixing, viscosity, and solvent concentration to produce the desired final paint product. In this report, the mathematical model is derived, controllers are tuned, and both continuous and discrete simulation results (using MATLAB) are validated. In addition, a comprehensive Simulink implementation is developed and tested.

2 Units and Dimensions

For consistency and physical relevance, the following units are used:

- **Pigment Concentrations** (R, G, B): $\mathbf{kg/m^3}$.
- **Viscosity** (V): $\mathbf{Pa \cdot s}$ (Pascal-seconds).
- **Solvent Concentration** (S): Dimensionless mass fraction (0–1, where 1 corresponds to 100%).

3 Model Derivation

The dynamics of the paint production process are described by a set of ordinary differential equations (ODEs) that capture the interactions between pigment mixing, viscosity, and solvent concentration. The state vector is defined as:

$$x = \begin{bmatrix} R \\ G \\ B \\ V \\ S \end{bmatrix},$$

The control input vector is defined as well as:

$$u = \begin{bmatrix} u_R \\ u_G \\ u_B \\ u_V \\ u_S \end{bmatrix}.$$

3.1 Color Mixing Dynamics

The mixing of primary colors is governed by:

$$\frac{dR}{dt} = \alpha_R\, u_R - \beta_R\, R - \gamma_{RV}\, V\, R - \gamma_{RS}\, S\, R,$$
$$\frac{dG}{dt} = \alpha_G\, u_G - \beta_G\, G - \gamma_{GV}\, V\, G - \gamma_{GS}\, S\, G,$$
$$\frac{dB}{dt} = \alpha_B\, u_B - \beta_B\, B - \gamma_{BV}\, V\, B - \gamma_{BS}\, S\, B,$$

where u_R, u_G, and u_B are the control inputs (flow rates) for the red, green, and blue pigments, respectively. The parameters α, β, and γ represent the input gains, natural degradation rates, and coupling effects with viscosity and solvent.

3.2 Viscosity Dynamics

Viscosity is influenced by pigment concentrations, a viscosity additive, and solvent concentration. It is modeled as:

$$\frac{dV}{dt} = -\delta_V\, V + \eta_V\, u_V + \kappa_R\, R + \kappa_G\, G + \kappa_B\, B - \lambda_V\, S\, V,$$

where:

- δ_V: Natural viscosity reduction rate.
- η_V: Effectiveness of the viscosity additive.
- κ_R, κ_G, κ_B: Influence of pigment concentrations on viscosity.
- λ_V: Viscosity reduction due to solvent S.

The control input u_V is provided by a relay controller.

3.3 Solvent Dynamics

The solvent concentration is governed by:

$$\frac{dS}{dt} = \alpha_S\, u_S - \beta_S\, S - \sigma_R\, R\, S - \sigma_G\, G\, S - \sigma_B\, B\, S - \epsilon_V\, V\, S,$$

where u_S is the control input for the solvent pump and the parameters denote the solvent input gain, evaporation rate, pigment absorption, and the effect of viscosity on solvent retention.

4 Controller Tuning Results

Two types of controllers are implemented:

4.1 PID Controllers for Color Mixing and Solvent Regulation

The PID controllers for the pigments and the solvent were tuned to achieve minimal overshoot and steady-state error. Initial gains were manually setted to:

- **Red PID Controller:** $K_p = 2.0$, $K_i = 0.5$, $K_d = 0.1$.
- **Green PID Controller:** $K_p = 2.0$, $K_i = 0.5$, $K_d = 0.1$.
- **Blue PID Controller:** $K_p = 2.0$, $K_i = 0.5$, $K_d = 0.1$.
- **Solvent PID Controller:** $K_p = 1.5$, $K_i = 0.5$, $K_d = 0.1$.

The PID controller is implemented as:

$$u = K_p e + K_i \int e\, dt + K_d \frac{de}{dt},$$

where e is the error between the setpoint and the current value.

4.2 Relay Controller for Viscosity

For viscosity, with a setpoint $V_{\text{ref}} = 1.2\,\text{Pa}\cdot\text{s}$, and using as setpoint for our pigments and solvent:

$$R_{\text{ref}} = 2.0\,\text{kg/m}^3, \quad G_{\text{ref}} = 0.5\,\text{kg/m}^3, \quad B_{\text{ref}} = 1.8\,\text{kg/m}^3, \quad S_{\text{ref}} = 0.7,$$

and parameters:

$$\delta_V = 0.5, \quad \eta_V = 0.8, \quad \kappa_R = 0.15, \quad \kappa_G = 0.12, \quad \kappa_B = 0.10, \quad \lambda_V = 0.2,$$

The relay controller with hysteresis is defined such that:

$$u = \begin{cases} u_{\text{Vmax}}, & \text{if } V < V_{ref} - V_\delta, \\ u_{\text{Vmin}}, & \text{if } V > V_{ref} + V_\delta, \\ \text{previous } u, & \text{otherwise.} \end{cases}$$

the steady-state input is computed as $u_V \approx 0.285$. To implement this, the relay controller is configured with:

- **Hysteresis Band:** $V_\delta = 0.1\,\text{Pa}\cdot\text{s}$ (switching when $V < 1.1\,\text{Pa}\cdot\text{s}$ or $V > 1.3\,\text{Pa}\cdot\text{s}$).
- **Relay Outputs:** $u_{V\max} = 0.32$ and $u_{V\min} = 0.28$.

5 Simulation Results (MATLAB)

The simulation is performed using both a continuous integration (via `ode45`) and a discrete Euler method. Time-domain plots for key variables are generated (e.g., pigment concentrations, viscosity, solvent concentration, control inputs). Below is an example figure placeholder:

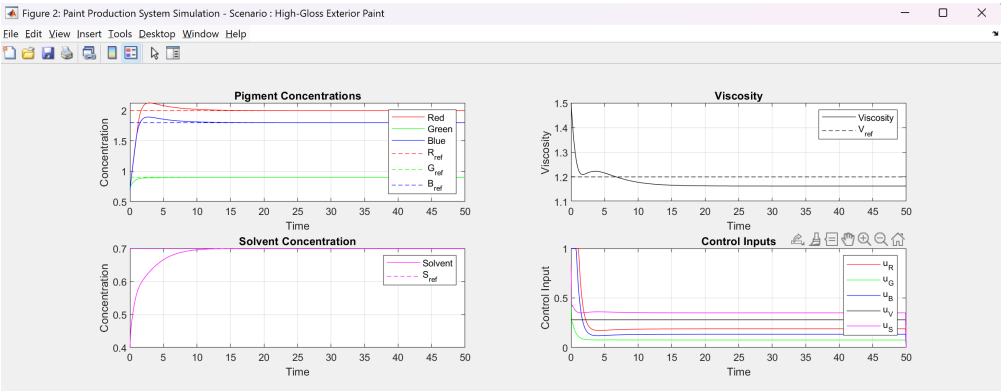

Figure 1: System dynamics over time (example from MATLAB simulation).

5.1 Final Output Color (MATLAB)

After the simulation, the final pigment concentrations are extracted and normalized to create an RGB color:

```
final_R = x_history(1, end);
final_G = x_history(2, end);
final_B = x_history(3, end);
maxVal = max([final_R, final_G, final_B]);
if maxVal > 0
    colorRGB = [final_R, final_G, final_B] / maxVal;
else
    colorRGB = [0, 0, 0];
end
```

A patch is then displayed with this color (see Figure 2).

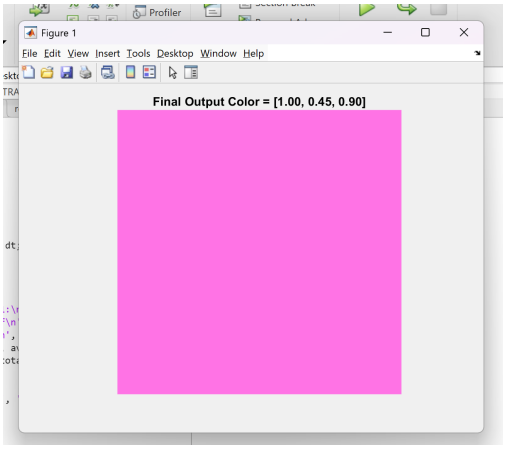

Figure 2: Final output color patch from MATLAB simulation.

6 Discretization

The continuous-time system is discretized using the Euler method. For a time step Δt, the state update is:

$$x(t + \Delta t) = x(t) + \Delta t \cdot \frac{dx}{dt}.$$

This discretization is validated by comparing it to the continuous solution obtained using `ode45`.

7 Validation of Discretization

To validate the Euler discretization, the absolute error between the continuous and discrete simulations is computed for each state variable. Let:

$$x_{i,\text{cont}}(t) \quad \text{and} \quad x_{i,\text{disc}}(t)$$

be the continuous and discrete solutions, respectively. The error is defined as:

$$\text{Error}_i(t) = \left| x_{i,\text{cont}}(t) - x_{i,\text{disc}}(t) \right|.$$

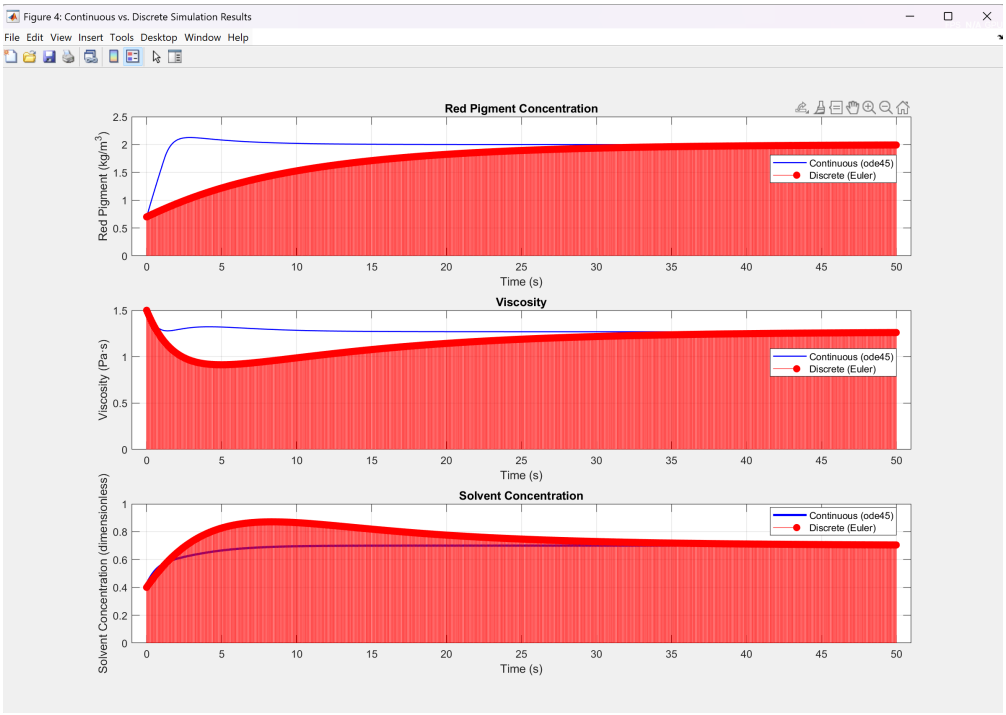

Figure 3: Validation between Continuous and Discrete time domains.

The MATLAB code used to compute and plot these errors is as follows:

```
% Compute the absolute error for each state variable
error_matrix = abs(x_history - x_discrete);

% Plot error for Red, Green, Blue, Viscosity, and Solvent
figure;
subplot(3,2,1);
plot(t, error_matrix(1,:), 'b-', 'LineWidth', 2);
xlabel('Time (s)');
```

```
ylabel('Error in R (kg/m^3)');
title('Error in Red Pigment');

subplot(3,2,2);
plot(t, error_matrix(2,:), 'g-', 'LineWidth', 2);
xlabel('Time (s)');
ylabel('Error in G (kg/m^3)');
title('Error in Green Pigment');

subplot(3,2,3);
plot(t, error_matrix(3,:), 'r-', 'LineWidth', 2);
xlabel('Time (s)');
ylabel('Error in B (kg/m^3)');
title('Error in Blue Pigment');

subplot(3,2,4);
plot(t, error_matrix(4,:), 'k-', 'LineWidth', 2);
xlabel('Time (s)');
ylabel('Error in V (Pa s)');
title('Error in Viscosity');

subplot(3,2,5);
plot(t, error_matrix(5,:), 'm-', 'LineWidth', 2);
xlabel('Time (s)');
ylabel('Error in S (dimensionless)');
title('Error in Solvent Concentration');
```

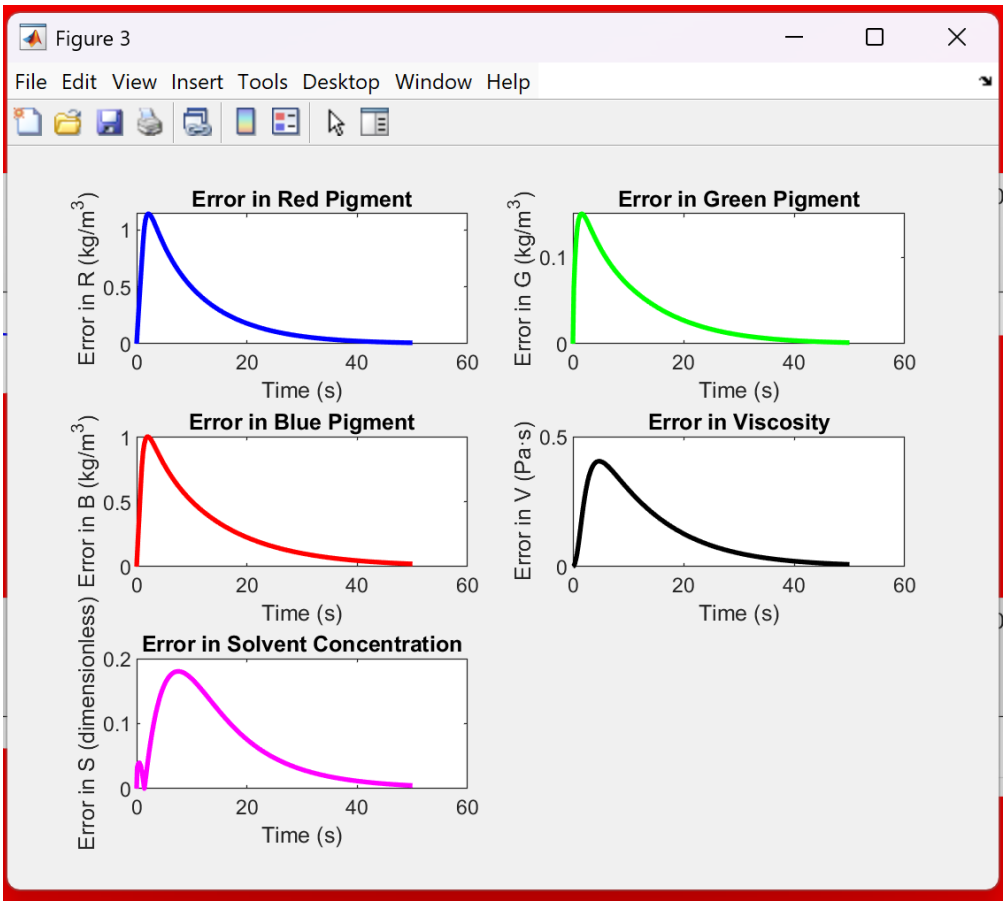

Figure 4: Absolute error in each state variable over time.

- **Maximum error in R: 1.1503**kg/m^3
- **Maximum error in G: 0.1524** kg/m^3
- **Maximum error in B: 1.0119** kg/m^3
- **Maximum error in V: 0.4034** Pas
- **Maximum error in S: 0.1844**

As shown in Figure 4, the error generally decreases to near zero, indicating that the Euler discretization (with the chosen $\Delta t = 0.06s$) is sufficiently accurate compared to the continuous simulation.

8 Simulink Implementation and Testing

In addition to the MATLAB simulation, a Simulink model was developed to implement the continuous dynamics and controllers in a graphical environment. The Simulink model was built as follows:

8.1 Model Overview

The Simulink model consists of:

- A **MATLAB Function block** that encapsulates the `paint_system_ode` function.
- An **Integrator** block that integrates $\dot{x}$ to yield the state x through its initial condition x_0.
- **PID Controllers** are used for precise regulation of the pigment flow rates (for red, green, and blue) and for solvent injection. Each PID controller compares the current measured value of a variable (for example, pigment concentration or solvent concentration) with its setpoint, computes the error, and then generates a control signal that minimizes this error over time.

- A **Relay Controller** block for viscosity. The Relay is configured with a hysteresis band so that:
  - When the measured viscosity is below $1.1\,\mathrm{Pa}\cdot\mathrm{s}$, the relay outputs a high value ($u_{V,\max} = 0.32$).
  - When the viscosity is above $1.3\,\mathrm{Pa}\cdot\mathrm{s}$, the relay outputs a low value ($u_{V,\min} = 0.28$). Essentially, the relay ensures that the viscosity is maintained within a narrow range around its setpoint.
- A **Mux/Demux** arrangement to combine and route signals between the plant, controllers, and integrator.

8.2 Implementation Details

1. **System:** The system is implemented using a MATLAB Function block that calls the `paint_system_ode` function. The block is set to have two input ports (for x and u) and one output port (for $\dot{x}$). An external Integrator block then updates the state x.

2. **Controllers:** The PID blocks are configured with the gains as specified in Section 6. The Relay block for viscosity is configured with the thresholds as described earlier. If necessary, an error signal (e.g., $V_{\text{ref}} - V$) is computed before the Relay to obtain the desired control logic.

3. **Signal Routing:** Mux and Demux blocks are used to combine the control signals into a single vector $u = [u_R; u_G; u_B; u_V; u_S]$ that is fed into the plant subsystem. Similarly, the state vector x is demultiplexed and fed back to each controller.

4. **Parameter Passing:** The model uses parameters (such as α_R, β_R, etc.) defined in the MATLAB workspace via an initialization script :

   - Add this line to MatLAB shell to link "params" to constant block for Simulink further usage
   - busInfo = Simulink.Bus.createObject(params);
   - busObj = evalin('base', busInfo.busName);

   After executing the script, we have setted a constant block which takes as input the busObs containing all the parameters initialized in the init.m file.

5. **Solver Configuration:** The model is configured with a fixed-step solver when a discrete-time implementation is desired, or a variable-step solver for continuous simulation. The sample time is set to match the desired Δt which in our case was $0.1s$.

8.3 Testing and Results

The Simulink model was simulated over a time period matching the MATLAB simulation. Key outputs (pigment concentrations, viscosity, and solvent concentration) were observed using Scope blocks and logged to the workspace. The simulation results showed that:

- All state variables converge smoothly toward their setpoints.
- The PID controllers for the pigments and solvent provide a fast and stable response.
- The Relay controller for viscosity effectively maintains the viscosity near the setpoint with minimal chattering.

Figure 5 shows a screenshot of the Simulink block diagram, and Figure 6 shows typical scope outputs from the Simulink simulation.

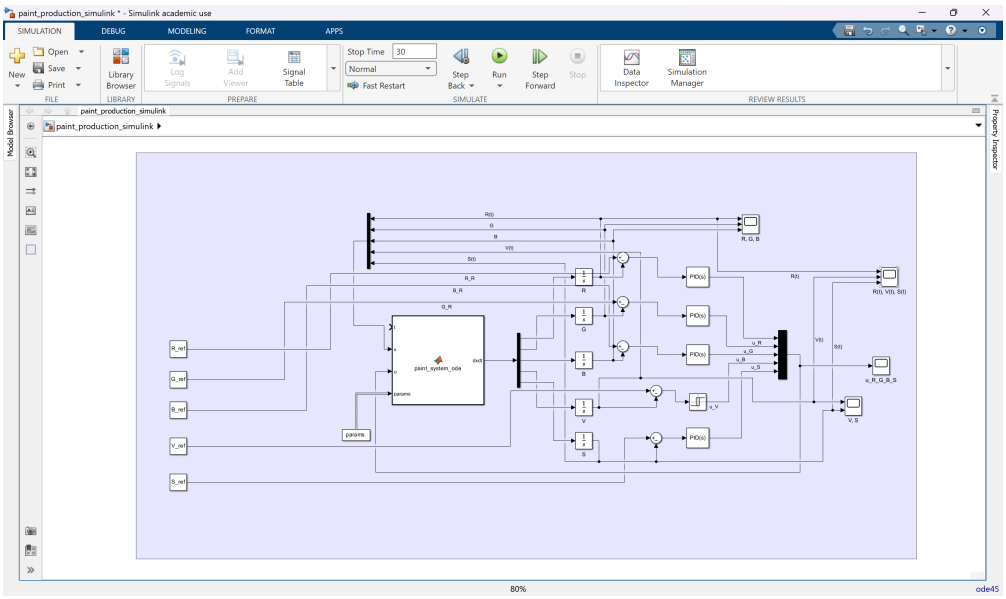

Figure 5: Simulink block diagram of the paint production system.

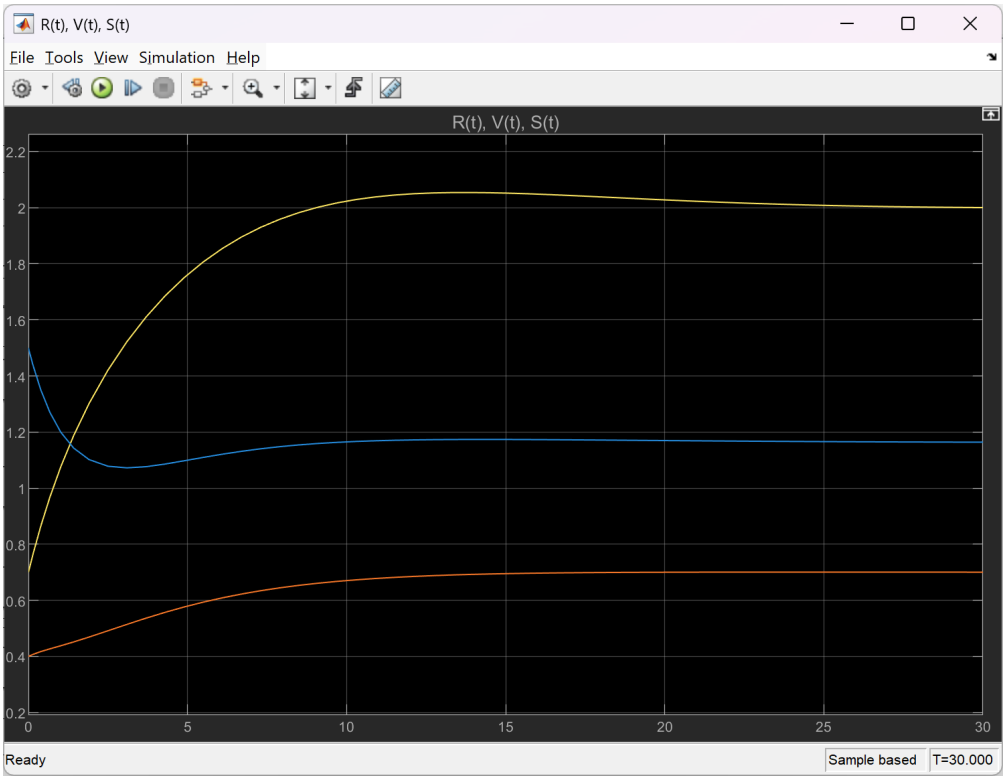

Figure 6: System dynamics over time: R(t), V(t), S(t).

8.4 Tuning for PID Controllers

The tuning process for the PID controllers was performed as follows:

- The parameters were fine-tuned through the Simlulink tool in the simulation to reduce overshoot,

improve settling time, and minimize steady-state error.

- Figures 7, 8, 9, and 10 display the final tuned PID blocks for the red, green, blue pigments and the solvent, respectively.

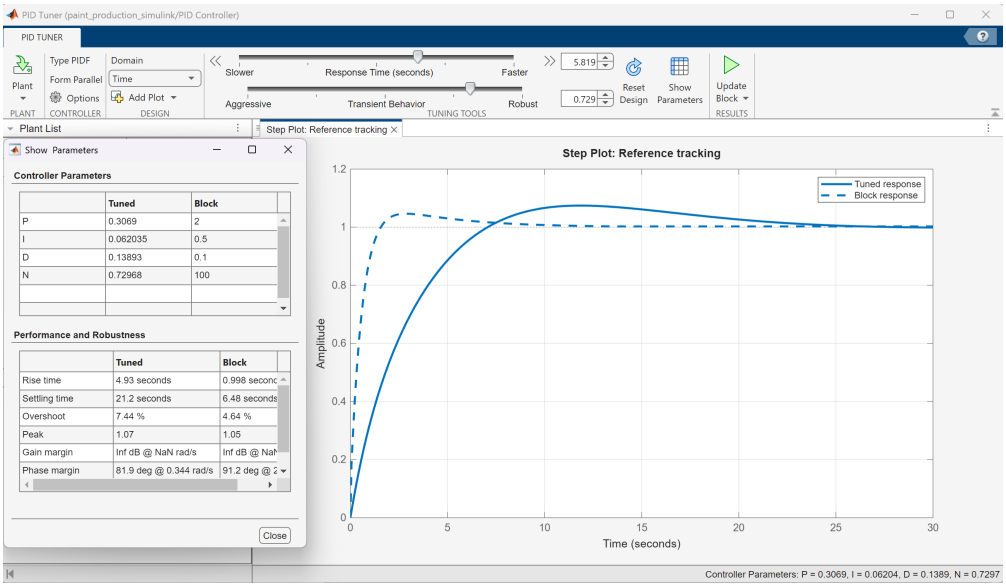

Figure 7: PID Controller for Red Pigment.

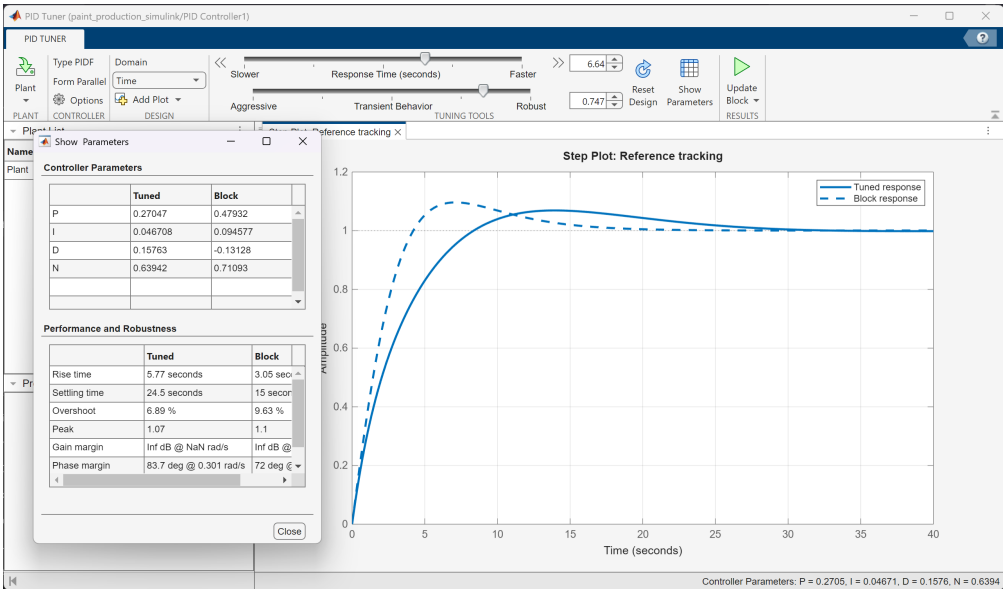

Figure 8: PID Controller for Green Pigment.

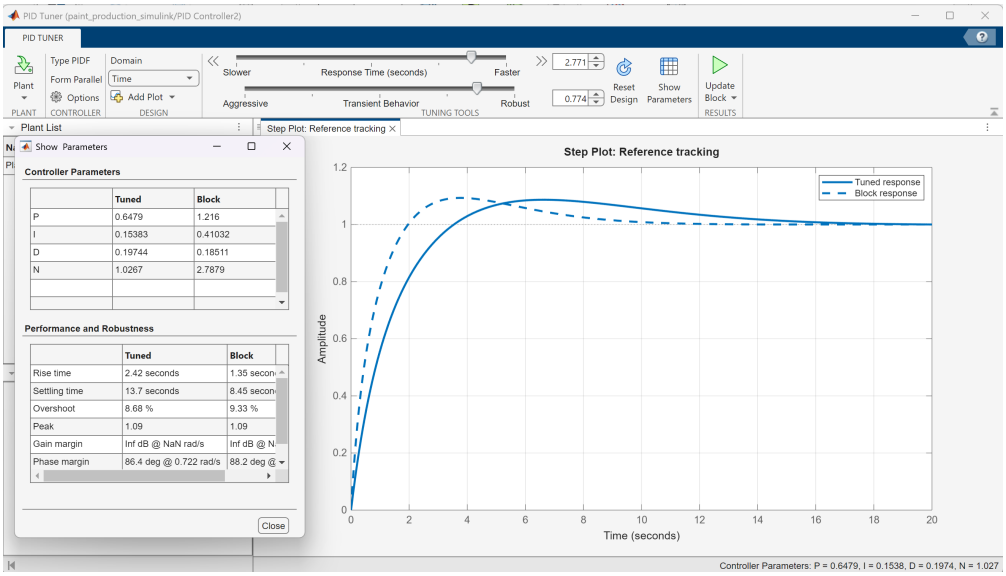

Figure 9: PID Controller for Blue Pigment.

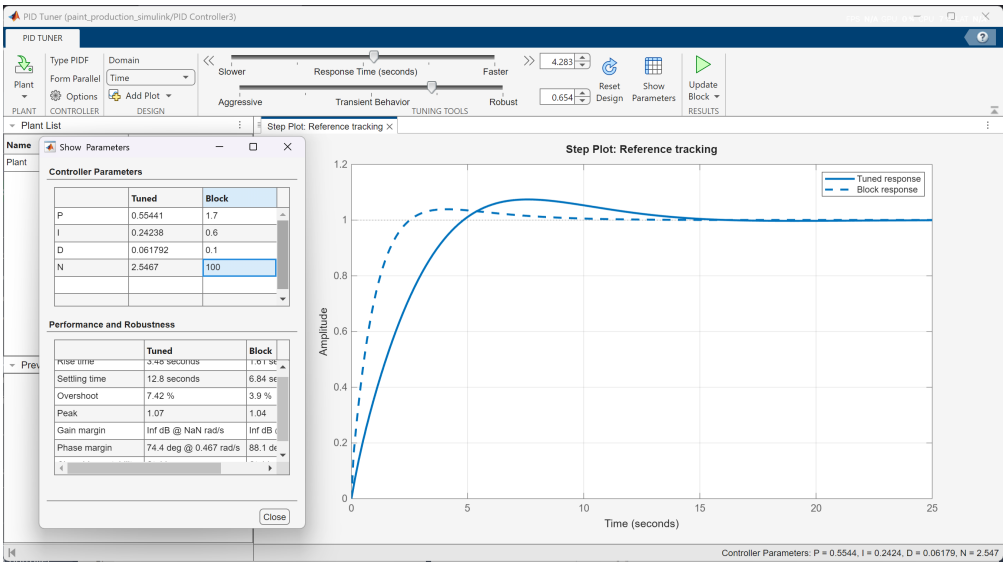

Figure 10: PID Controller for Solvent.

This systematic tuning ensured that the control loops respond quickly without excessive oscillation and maintain the process variables at their desired setpoints.

9 Sustainability Analysis

Sustainability is evaluated using three key metrics:

9.1 Material Efficiency

- **Metric:** $\dfrac{\text{Pigment Used}}{\text{Final Paint Output}}$

- **Value:** 4.2344

This metric indicates that for every unit of pigment used, the final output is amplified by a factor of 4.2344.

9.2 Energy Efficiency

- **Metric:** $\frac{\text{Mixing Energy Consumption}}{\text{Final Paint Output}}$
- **Value:** 1.99801

A lower ratio suggests efficient energy use during the mixing process.

9.3 Solvent Environmental Impact

- **Metric:** $\frac{\text{Solvent Emission}}{\text{Regulatory Limit}}$
- **Regulatory Limit:** 10% (assumed)
- **Value:** 1.22461

This metric assesses whether the solvent emissions comply with regulatory standards.

A MATLAB Code Listings

A.1 Initialization: `init.m`

```
% init.m - Parameter initialization for paint production system - Scenario : High-Gloss Ex
% This file contains all parameters needed for the model

% Simulation parameters
tspan = [0 50];  % Simulation time span [start end]
dt = 0.1;        % Time step for simulation
dt2 = 0.06;

% Initial conditions
R0 = 0.7;        % Initial red pigment concentration
G0 = 0.7;        % Initial green pigment concentration
B0 = 0.7;        % Initial blue pigment concentration
V0 = 1.5;        % Initial viscosity
S0 = 0.4;        % Initial solvent concentration
initial_state = [R0; G0; B0; V0; S0];

% Reference values (setpoints) - Scenario 2: High-Gloss Exterior Paint
R_ref = 2.0;     % High red pigment concentration
G_ref = 0.9;     % Low green pigment concentration
B_ref = 1.8;     % High blue pigment concentration
V_ref = 1.2;     % Lower viscosity (thinner paint)
S_ref = 0.7;     % Higher solvent concentration (faster drying)

% Color mixing parameters
alpha_R = 1.0;   % Input gain for red pigment
alpha_G = 1.0;   % Input gain for green pigment
alpha_B = 1.0;   % Input gain for blue pigment
beta_R = 0.05;   % Natural degradation rate for red pigment
beta_G = 0.04;   % Natural degradation rate for green pigment
beta_B = 0.03;   % Natural degradation rate for blue pigment

% Coupling coefficients between pigments and viscosity
gamma_RV = 0.02;  % Effect of viscosity on red pigment
gamma_GV = 0.02;  % Effect of viscosity on green pigment
gamma_BV = 0.02;  % Effect of viscosity on blue pigment

% Coupling coefficients between pigments and solvent
gamma_RS = 0.03;  % Effect of solvent on red pigment
gamma_GS = 0.03;  % Effect of solvent on green pigment
gamma_BS = 0.03;  % Effect of solvent on blue pigment

% Viscosity parameters
delta_V = 0.51;    % Natural viscosity reduction rate
eta_V = 0.6;     % Viscosity additive effectiveness
kappa_R = 0.15;  % Impact of red pigment on viscosity
kappa_G = 0.12;  % Impact of green pigment on viscosity
kappa_B = 0.10;  % Impact of blue pigment on viscosity
lambda_V = 0.2;  % Solvent's effect on reducing viscosity

% Solvent parameters
```

```
alpha_S = 0.7;    % Solvent input gain
beta_S = 0.08;    % Solvent evaporation rate
sigma_R = 0.05;   % Absorption rate of solvent by red pigment
sigma_G = 0.05;   % Absorption rate of solvent by green pigment
sigma_B = 0.05;   % Absorption rate of solvent by blue pigment
epsilon_V = 0.03; % Effect of viscosity on solvent retention

% PID controller parameters for red pigment
Kp_R = 2.0;       % Proportional gain
Ki_R = 0.5;       % Integral gain
Kd_R = 0.1;       % Derivative gain

% PID controller parameters for green pigment
Kp_G = 2.0;       % Proportional gain
Ki_G = 0.5;       % Integral gain
Kd_G = 0.1;       % Derivative gain

% PID controller parameters for blue pigment
Kp_B = 2.0;       % Proportional gain
Ki_B = 0.5;       % Integral gain
Kd_B = 0.1;       % Derivative gain

% Relay controller parameters for viscosity
V_delta = 0.2;    % Hysteresis band
u_V_max = 0.32;    % Maximum viscosity additive input
u_V_min = 0.28;    % Minimum viscosity additive input

% PID controller parameters for solvent
Kp_S = 1.7;       % Proportional gain
Ki_S = 0.6;       % Integral gain
Kd_S = 0.1;       % Derivative gain

% Energy and material consumption parameters
energy_factor_R = 0.1;  % Energy consumption factor for red pigment pump
energy_factor_G = 0.1;  % Energy consumption factor for green pigment pump
energy_factor_B = 0.1;  % Energy consumption factor for blue pigment pump
energy_factor_V = 0.2;  % Energy consumption factor for viscosity additive
energy_factor_S = 0.15; % Energy consumption factor for solvent pump

% Display initialization message
disp('Paint production system parameters initialized for Scenario : High-Gloss Exterior Pa
```

A.2 Paint System ODE: `paint_system_ode.m`

```
function dxdt = paint_system_ode(t, x, u, params)
% PAINT_SYSTEM_ODE - ODE function for paint production system
%
% Inputs:
%   t - current time
%   x - current state [R; G; B; V; S]
%   u - control inputs [u_R; u_G; u_B; u_V; u_S]
%   params - structure containing all model parameters
%
% Outputs:
```

```
%   dxdt - state derivatives [dR/dt; dG/dt; dB/dt; dV/dt; dS/dt]

% Extract states
R = x(1);  % Red pigment concentration
G = x(2);  % Green pigment concentration
B = x(3);  % Blue pigment concentration
V = x(4);  % Viscosity
S = x(5);  % Solvent concentration

% Extract control inputs
u_R = u(1);  % Red pigment flow rate
u_G = u(2);  % Green pigment flow rate
u_B = u(3);  % Blue pigment flow rate
u_V = u(4);  % Viscosity additive input
u_S = u(5);  % Solvent flow rate

% Color mixing dynamics
dRdt = params.alpha_R * u_R - params.beta_R * R - params.gamma_RV * V * R - params.gamma_RS
dGdt = params.alpha_G * u_G - params.beta_G * G - params.gamma_GV * V * G - params.gamma_GS
dBdt = params.alpha_B * u_B - params.beta_B * B - params.gamma_BV * V * B - params.gamma_BS

% Viscosity dynamics
dVdt = -params.delta_V * V + params.eta_V * u_V + ...
       params.kappa_R * R + params.kappa_G * G + params.kappa_B * B - ...
       params.lambda_V * S * V;

% Solvent concentration dynamics
dSdt = params.alpha_S * u_S - params.beta_S * S - ...
       params.sigma_R * R * S - params.sigma_G * G * S - params.sigma_B * B * S - ...
       params.epsilon_V * V * S;

% Combine state derivatives
dxdt = [dRdt; dGdt; dBdt; dVdt; dSdt];
end
```

A.3 PID Controller: `pid_controller.m`

```
function [u, error_int] = pid_controller(reference, current, error_int, dt, Kp, Ki, Kd, pre
% PID_CONTROLLER - Implements a PID controller
%
% Inputs:
%   reference - setpoint value
%   current - current process value
%   error_int - integral of error (state)
%   dt - time step
%   Kp - proportional gain
%   Ki - integral gain
%   Kd - derivative gain
%   prev_error - previous error for derivative calculation
%
% Outputs:
%   u - control signal
%   error_int - updated integral of error
```

```
% Calculate error
error = reference - current;

% Update integral term
error_int = error_int + error * dt;

% Calculate derivative term
if nargin < 8
    error_deriv = 0;
else
    error_deriv = (error - prev_error) / dt;
end

% Calculate control signal
u = Kp * error + Ki * error_int + Kd * error_deriv;

% Limit control signal to prevent excessive values
u = max(0, min(u, 1));  % Assuming control range is [0, 1]
end
```

A.4 Relay Controller: `relay_controller.m`

```
function u = relay_controller(reference, current, delta, u_max, u_min, prev_u)
% RELAY_CONTROLLER - Implements a relay controller with hysteresis
%
% Inputs:
%   reference - setpoint value
%   current - current process value
%   delta - hysteresis band
%   u_max - maximum control output
%   u_min - minimum control output
%   prev_u - previous control output
%
% Outputs:
%   u - control signal

if current < reference - delta
    u = u_max;
elseif current > reference + delta
    u = u_min;
else
    u = prev_u;  % Maintain previous value within hysteresis band
end
end
```

A.5 Discretization Script: `discretization.m`

```
% Define discrete time vector (using the same tspan and dt as before)
t_discrete = tspan(1):dt2:tspan(2);

% Initialize the discrete state matrix.
% Assuming initial_state is a column vector representing the state at time t = 0.
x_discrete = zeros(length(initial_state), length(t_discrete));
x_discrete(:, 1) = initial_state;
```

```
% Euler integration loop
for k = 1:length(t_discrete)-1
    % Compute the state derivative at the current discrete time step
    dx = paint_system_ode(0, x_discrete(:, k), u, params);

    % Update the state using Euler's method
    x_discrete(:, k+1) = x_discrete(:, k) + dt2 * dx;
end

% Calculate the error matrix between continuous and discrete solutions
% x_history and x_discrete are both of size [number_of_states x num_steps]

x_history_interp = interp1(t, x_history', t_discrete)';
error_matrix = abs(x_history_interp - x_discrete);
% Plot the error for each state variable


figure;

% Red Pigment (R)
subplot(3,2,1);
plot(t_discrete, error_matrix(1, :), 'b-', 'LineWidth', 2);
xlabel('Time (s)');
ylabel('Error in R (kg/m^3)');
title('Error in Red Pigment');

% Green Pigment (G)
subplot(3,2,2);
plot(t_discrete, error_matrix(2, :), 'g-', 'LineWidth', 2);
xlabel('Time (s)');
ylabel('Error in G (kg/m^3)');
title('Error in Green Pigment');

% Blue Pigment (B)
subplot(3,2,3);
plot(t_discrete, error_matrix(3, :), 'r-', 'LineWidth', 2);
xlabel('Time (s)');
ylabel('Error in B (kg/m^3)');
title('Error in Blue Pigment');

% Viscosity (V)
subplot(3,2,4);
plot(t_discrete, error_matrix(4, :), 'k-', 'LineWidth', 2);
xlabel('Time (s)');
ylabel('Error in V (Pa s)');
title('Error in Viscosity');

% Solvent Concentration (S)
subplot(3,2,5);
plot(t_discrete, error_matrix(5, :), 'm-', 'LineWidth', 2);
xlabel('Time (s)');
ylabel('Error in S (dimensionless)');
title('Error in Solvent Concentration');
```

```
% Optionally, display maximum error for each state in the command window:
fprintf('Maximum error in R: %.4f\n', max(error_matrix(1, :)));
fprintf('Maximum error in G: %.4f\n', max(error_matrix(2, :)));
fprintf('Maximum error in B: %.4f\n', max(error_matrix(3, :)));
fprintf('Maximum error in V: %.4f\n', max(error_matrix(4, :)));
fprintf('Maximum error in S: %.4f\n', max(error_matrix(5, :)));

save('paint_system_results_scenario_dics_VS_contnuos.mat', 'x_discrete', 'x_history', 'erro



% Assume:
% - 't' is the time vector for the continuous simulation,
% - 'x_history' contains the continuous simulation states [R; G; B; V; S],
% - 't_discrete' is the discrete time vector,
% - 'x_discrete' contains the discrete simulation states [R; G; B; V; S].

figure('Name', 'Continuous vs. Discrete Simulation Results', 'Position', [100, 100, 1200, 8

% Plot Red Pigment (R)
subplot(3,1,1);
plot(t, x_history(1, :), 'b-', 'LineWidth', 1);
hold on;
stem(t_discrete, x_discrete(1, :), 'ro', 'filled');
xlabel('Time (s)');
ylabel('Red Pigment (kg/m^3)');
legend('Continuous (ode45)', 'Discrete (Euler)', 'Location', 'Best');
title('Red Pigment Concentration');
grid on;
hold off;

% Plot Viscosity (V)
subplot(3,1,2);
plot(t, x_history(4, :), 'b-', 'LineWidth', 1);
hold on;
stem(t_discrete, x_discrete(4, :), 'ro', 'filled');
xlabel('Time (s)');
ylabel('Viscosity (Pa s)');
legend('Continuous (ode45)', 'Discrete (Euler)', 'Location', 'Best');
title('Viscosity');
grid on;
hold off;

% Plot Solvent Concentration (S)
subplot(3,1,3);
plot(t, x_history(5, :), 'b-', 'LineWidth', 2);
hold on;
stem(t_discrete, x_discrete(5, :), 'ro', 'filled');
xlabel('Time (s)');
ylabel('Solvent Concentration (dimensionless)');
legend('Continuous (ode45)', 'Discrete (Euler)', 'Location', 'Best');
title('Solvent Concentration');
grid on;
hold off;
```

A.6 Simulation Script: `run.m`

```
% run.m - Main script to run the paint production system simulation - Scenario : High-Gloss

% Clear workspace and close figures
clear;
clc;
close all;

% Initialize parameters
init;

% Convert parameters to structure for easier passing
params.alpha_R = alpha_R;
params.alpha_G = alpha_G;
params.alpha_B = alpha_B;
params.beta_R = beta_R;
params.beta_G = beta_G;
params.beta_B = beta_B;
params.gamma_RV = gamma_RV;
params.gamma_GV = gamma_GV;
params.gamma_BV = gamma_BV;
params.gamma_RS = gamma_RS;
params.gamma_GS = gamma_GS;
params.gamma_BS = gamma_BS;
params.delta_V = delta_V;
params.eta_V = eta_V;
params.kappa_R = kappa_R;
params.kappa_G = kappa_G;
params.kappa_B = kappa_B;
params.lambda_V = lambda_V;
params.alpha_S = alpha_S;
params.beta_S = beta_S;
params.sigma_R = sigma_R;
params.sigma_G = sigma_G;
params.sigma_B = sigma_B;
params.epsilon_V = epsilon_V;

% Time vector for simulation
t = tspan(1):dt:tspan(2);
num_steps = length(t);

% Initialize state and control history arrays
x_history = zeros(5, num_steps);
u_history = zeros(5, num_steps);

% Set initial state
x = initial_state;
x_history(:, 1) = x;

% Initialize controller states
error_int_R = 0;
error_int_G = 0;
error_int_B = 0;
```

```
error_int_S = 0;
prev_error_R = 0;
prev_error_G = 0;
prev_error_B = 0;
prev_error_S = 0;
prev_u_V = 0;

%Main simulation loop
for i = 1:num_steps-1
    % Current state
    R = x(1);
    G = x(2);
    B = x(3);
    V = x(4);
    S = x(5);

    % PID controller for red pigment
    [u_R, error_int_R] = pid_controller(R_ref, R, error_int_R, dt, Kp_R, Ki_R, Kd_R, prev_e
    prev_error_R = R_ref - R;

    % PID controller for green pigment
    [u_G, error_int_G] = pid_controller(G_ref, G, error_int_G, dt, Kp_G, Ki_G, Kd_G, prev_e
    prev_error_G = G_ref - G;

    % PID controller for blue pigment
    [u_B, error_int_B] = pid_controller(B_ref, B, error_int_B, dt, Kp_B, Ki_B, Kd_B, prev_e
    prev_error_B = B_ref - B;

    % Relay controller for viscosity
    u_V = relay_controller(V_ref, V, V_delta, u_V_max, u_V_min, prev_u_V);
    prev_u_V = u_V;

    % PID controller for solvent
    [u_S, error_int_S] = pid_controller(S_ref, S, error_int_S, dt, Kp_S, Ki_S, Kd_S, prev_e
    prev_error_S = S_ref - S;

    % Combine control inputs
    u = [u_R; u_G; u_B; u_V; u_S];
    u_history(:, i) = u;
    % Computing Mixing Efficiency
    pigment_rate = params.alpha_R*u_R + params.alpha_G*u_G + params.alpha_B*u_B;
    total_pigment_used = trapz(t, pigment_rate * ones(size(t)));

    % Calculate energy consumption
    energy_rate = energy_factor_R*u_R^2 + energy_factor_G*u_G^2 + energy_factor_B*u_B^2 +
              energy_factor_V*u_V^2 + energy_factor_S*u_S^2;

    % Calculate Mixing Energy
    mixing_energy = trapz(t, energy_rate * ones(size(t)));

    % Calculate Solvent Emissions
    total_solvent_injected = trapz(t, params.alpha_S*u_S * ones(size(t)));
    solvent_emissions = total_solvent_injected - x_history(5,end);
    regulatory_limit = 10;  % Regulatory Limit chosen arbitrarily
```

```
    % Simulate system for one time step using ODE45
    [~, x_step] = ode45(@(t, x) paint_system_ode(t, x, u, params), [0 dt], x);

    % Update state
    x = x_step(end, :)';
    x_history(:, i+1) = x;
end

% — After your simulation finishes:
final_R = x_history(1, end);
final_G = x_history(2, end);
final_B = x_history(3, end);

% — Normalize to [0, 1] range by dividing by the max component to get the final RGB vecto
maxVal = max([final_R, final_G, final_B]);
if maxVal > 0
    colorRGB = [final_R, final_G, final_B] / maxVal;
else
    colorRGB = [0, 0, 0];
end

% Compute the Mixing Ratio Efficiency, Energy Efficiency and Environmental Impact:
final_paint_output = final_R + final_G + final_B;
material_efficiency = total_pigment_used / final_paint_output;
env_impact = solvent_emissions / regulatory_limit;
energy_efficiency = mixing_energy / final_paint_output;

% — Display in a figure:
figure;
patch([0 1 1 0], [0 0 1 1], colorRGB, 'EdgeColor', 'none');
axis equal; axis off;
title(sprintf('Final Output Color = [%.2f, %.2f, %.2f]', colorRGB));


% Plot results
figure('Name', 'Paint Production System Simulation - Scenario : High-Gloss Exterior Paint'

% Plot pigment concentrations
subplot(3, 2, 1);
plot(t, x_history(1, :), 'r-', t, x_history(2, :), 'g-', t, x_history(3, :), 'b-');
hold on;
plot(t, R_ref * ones(size(t)), 'r--', t, G_ref * ones(size(t)), 'g--', t, B_ref * ones(size
hold off;
xlabel('Time');
ylabel('Concentration');
title('Pigment Concentrations');
legend('Red', 'Green', 'Blue', 'R_{ref}', 'G_{ref}', 'B_{ref}');
grid on;

% Plot viscosity
subplot(3, 2, 2);
plot(t, x_history(4, :), 'k-', t, V_ref * ones(size(t)), 'k--');
xlabel('Time');
```

```
ylabel('Viscosity');
title('Viscosity');
legend('Viscosity', 'V_{ref}');
grid on;

% Plot solvent concentration
subplot(3, 2, 3);
plot(t, x_history(5, :), 'm-', t, S_ref * ones(size(t)), 'm--');
xlabel('Time');
ylabel('Concentration');
title('Solvent Concentration');
legend('Solvent', 'S_{ref}');
grid on;

% Plot control inputs
subplot(3, 2, 4);
plot(t, u_history(1, :), 'r-', t, u_history(2, :), 'g-', t, u_history(3, :), 'b-', ...
     t, u_history(4, :), 'k-', t, u_history(5, :), 'm-');
xlabel('Time');
ylabel('Control Input');
title('Control Inputs');
legend('u_R', 'u_G', 'u_B', 'u_V', 'u_S');
grid on;


% Display performance metrics
fprintf('Performance Metrics - Scenario : High-Gloss Exterior Paint\n');

fprintf('Material Total Pigment Used: %.4f\n', total_pigment_used);
fprintf('Material Final Paint output: %.4f\n', final_paint_output);
fprintf('Material Efficiency : %.4f\n', material_efficiency);
fprintf('Mixing Energy: %.4f1\n', mixing_energy);
fprintf('Solvent Environments Impact: %.4f1\n', env_impact);

% Save results
save('paint_system_results_scenario1.mat', 'x_history', 'u_history', 'total_pigment_used',
```

B Conclusion

This report presented the derivation of the mathematical model for sustainable paint production, including ODEs for pigment mixing, viscosity, and solvent concentration. Controller tuning for both PID and relay-based approaches was discussed, and simulation results were obtained using continuous (`ode45`) and discrete (Euler) methods in MATLAB. In addition, a comprehensive Simulink model was developed to implement and test the system in a graphical environment. The validation of the discretization and the Simulink simulation results demonstrate that the system operates as expected and converges to the desired setpoints. Overall, the integrated MATLAB and Simulink approaches confirm the robustness of the proposed control strategy.